

Clases genéricas

Esteban Álvarez

Métodos genéricos

- Revisa el siguiente método estático de la clase `Imprimir`
- ¿Qué hace?

```
public static void imprimir (Integer numero) {  
    System.out.println(numero+" es de tipo: "+ numero.getClass().getName());  
}
```

Métodos genéricos

- Recibe un **objeto Integer** e imprime su valor y su tipo de datos
- Ejemplo de uso en el Main:

```
public static void main(String[] args) {  
    Integer num1=5;  
    Imprimir.imprimir(num1);  
}
```

```
5 es de tipo: java.lang.Integer
```

Métodos genéricos

- **PROBLEMA:** Haz que la clase Imprimir también haga la misma tarea para objetos de tipo Double y String

Métodos genéricos

- **PROBLEMA:** Haz que la clase Imprimir también haga la misma tarea para objetos de tipo Double y String

```
public static void imprimir (Integer numero) {  
    System.out.println(numero+" es de tipo: "+ numero.getClass().getName());  
}  
  
public static void imprimir (Double numero) {  
    System.out.println(numero+" es de tipo: "+ numero.getClass().getName());  
}  
  
public static void imprimir (String cadena) {  
    System.out.println(cadena+" es de tipo: "+ cadena.getClass().getName());  
}
```

Métodos genéricos

- El uso en el main sería llamando a cada método sobrecargado:

```
Integer num1=5;  
Imprimir.imprimir(num1);
```

```
Double num2=5.4;  
Imprimir.imprimir(num2);
```

```
String cadena="Hola";  
Imprimir.imprimir(cadena);
```

```
public static void imprimir (Integer numero) {  
    System.out.println(numero+" es de tipo: "+ numero.getClass().getName());  
}  
  
public static void imprimir (Double numero) {  
    System.out.println(numero+" es de tipo: "+ numero.getClass().getName());  
}  
  
public static void imprimir (String cadena) {  
    System.out.println(cadena+" es de tipo: "+ cadena.getClass().getName());  
}
```

```
5 es de tipo: java.lang.Integer  
5.4 es de tipo: java.lang.Double  
Hola es de tipo: java.lang.String
```

Métodos genéricos

- Pero se está **triplicando** este código!

```
public static void imprimir (Integer numero) {  
    System.out.println(numero+" es de tipo: "+ numero.getClass().getName());  
}
```

```
public static void imprimir (Double numero) {  
    System.out.println(numero+" es de tipo: "+ numero.getClass().getName());  
}
```

```
public static void imprimir (String cadena) {  
    System.out.println(cadena+" es de tipo: "+ cadena.getClass().getName());  
}
```

Métodos genéricos

- La mejor solución consiste en implementar **un único método** genérico
 - De forma que puede recibir un objeto de tipo Integer, Double o String
 - E independientemente del tipo de datos del parámetro, mostrar su valor y su clase

```
public static void imprimir (???? dato) {  
    System.out.println(dato+" es de tipo: "+ dato.getClass().getName());  
}
```


Métodos genéricos

- Eso se consigue utilizando tipos **genéricos**
- Le indicamos al método que puede recibir cualquier tipo de objeto y, sea cual sea ese tipo, siempre tiene que hacer la misma tarea

```
public static <T> void imprimir (T dato) {  
    System.out.println(dato+" es de tipo: "+ dato.getClass().getName());  
}
```

Indica que es un método que va a recibir objetos de tipo T (el tipo que sea)

dato es un parámetro de tipo T, o sea genérico

Al símbolo < > se le denomina **operador diamante**

Métodos genéricos

- Ahora en el main se llamaría siempre al mismo método

```
Integer num1=5;  
Imprimir.imprimir(num1);
```

```
Double num2=5.4;  
Imprimir.imprimir(num2);
```

```
String cadena="Hola";  
Imprimir.imprimir(cadena);
```

```
public static <T> void imprimir (T dato) {  
    System.out.println(dato+" es de tipo: "+ dato.getClass().getName());  
}
```



```
5 es de tipo: java.lang.Integer  
5.4 es de tipo: java.lang.Double  
Hola es de tipo: java.lang.String
```

Métodos genéricos

- Para comprender los genéricos, el operador diamante <> se puede interpretar como si el tipo de datos fuese **Object**
- Recuerda que todas las clases **heredan** de Object
- A partir de la versión 5 de Java se recomienda el uso de **genéricos** en lugar de Object

```
public static void imprimir (Object dato) {  
    System.out.println(dato+" es de tipo: "+ dato.getClass().getName());  
}
```

Métodos genéricos

EJERCICIO

Piensa en cómo hacer las siguientes modificaciones al método imprimir

- Devolver un String en lugar de mostrar por pantalla
- Hacer que sólo se puedan imprimir datos de números (por ejemplo no se podrían mostrar datos de un objeto String)

Métodos genéricos

- Los métodos genéricos pueden recibir **más de un parámetro**

EJERCICIO

- Crea un método que reciba dos objetos y diga si son o no de la misma clase

Cabecera del método:

```
public static <T,U> boolean tiposIguales(T objeto1, U objeto2)
```

Métodos genéricos

- Uno de los **objetivos** de los métodos genéricos es seguir el principio **DRY**: Don't Repeat Yourself
- También permite utilizar nuestros métodos en un ámbito más amplio

Métodos genéricos

EJERCICIOS

1. Crea la clase “Matrices” con el método genérico Máximo. Que recibe un vector genérico y devuelve su valor máximo
 - Sobrecarga el método anterior para calcular el máximo de una matriz genérica
2. Crea una clase con dos métodos estáticos para guardar y leer objetos de un fichero binario serializado
 - Método guardar. Recibe una colección con objetos de cualquier tipo (genéricos) y guarda los objetos individualmente en el fichero cuya ruta especifica el usuario
 - Método leer. Lee los objetos de un fichero cuya ruta especifica el usuario y devuelve una colección genérica de esos objetos

Clases genéricas

- También se pueden usar genéricos con **clases**
- Es habitual cuando es necesario que nuestra clase **maneje cualquier tipo** de datos
- Ya **usamos** clases genéricas:
 - ArrayList
 - HashMap
 - HashSet

Clases genéricas

- Ejemplo de clase genérica que almacena cualquier objeto

```
public class ContenedorGenerico<T> {  
    private T dato;  
  
    public ContenedorGenerico(T dato) {  
        this.dato = dato;}  
  
    public T getDato() {  
        return dato;}  
  
    public void setDato(T dato) {  
        this.dato = dato;}  
    @Override  
    public String toString() {  
        return "ContenedorGenerico [dato=" + dato + "]";  
    }  
}
```

Clases genéricas

- Para crear un objeto de la clase anterior sólo tenemos que recordar cómo creamos, por ejemplo, un ArrayList:

```
ContenedorGenerico<Integer> entero= new ContenedorGenerico<>(5);  
ContenedorGenerico<Persona> persona= new ContenedorGenerico<>(new Persona());
```

Clases genéricas

EJERCICIO

- Crea la clase “Pareja” que guarde pares de datos. Ejemplo: “Lunes” y 1
- Implementa métodos un constructor con parámetros, métodos get para obtener cada uno de los valores y el método toString
- Modifica la clase anterior para que los datos que se almacenen **deban** ser del mismo tipo. string:string, integer:integer, etc.

Clases genéricas

EJERCICIO

- Crea la clase “RegistroParking” con tres atributos:
 - vehículo. De tipo genérico para poder registrar cualquier vehículo en el parking
 - horaEntrada y horaSalida de tipo LocalTime
- El constructor debe recibir el vehículo y la hora de entrada
- Implementa el método setHoraSalida y los métodos get para todos los atributos
- Implementa el método calcularRecibo que devuelve el precio a pagar por el vehículo a razón de 0,01345 €/minuto
- Crea la clase GestiónParking que maneje una colección de Registros correspondiente a un parking

Interfaces genéricas

- Las **interfaces** también pueden hacer uso de genéricos en la declaración de sus métodos

```
public interface Repetidor <T>{  
    public abstract void repetir(T dato);  
}
```

FIN!